

MANUAL TÉCNICO

MercaCredit

Sistema de Gestión de Cupos

Integrantes:

Mariana Camila Durán Piñeros
Mayerly Katherine Murcia Acosta
Gabriel Santiago Amortegui Ramírez

Curso: Base de Datos 1

Fecha: 14/07/2026

1. Introducción

Este manual describe la arquitectura, el modelo de datos, la estructura del código y los endpoints de la API REST del sistema MercaCredit. Esta es una aplicación de tres capas: un frontend construido en Angular, un backend de API REST construido en Spring Boot, y una base de datos relacional PostgreSQL. El sistema gestiona cupos de crédito para Clientes de una cadena de supermercados, quienes pueden repartir su cupo entre Parejas autorizadas, con control de restricciones horarias, historial de compras, y un flujo de solicitudes de sobrecupo con distintos niveles de aprobación (Cliente y Supervisor).

2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor de tres capas, comunicadas mediante peticiones HTTP en formato JSON:

- Frontend (Angular 21): aplicación que corre en el navegador, en `http://localhost:4200`. Consume la API REST del backend mediante `HttpClient`.
- Backend (Spring Boot 4.1.0): expone una API REST en `http://localhost:8585`, con el contexto raíz `"/compra"`.
- Base de datos (PostgreSQL 18): almacena los datos persistentes. El acceso se realiza mediante `JdbcTemplate` con SQL explícito, sin un framework ORM (no se usa JPA/Hibernate).

Dentro del backend, cada capa tiene una responsabilidad concreta:

- Controller: recibe la petición HTTP, valida el formato de entrada (anotaciones de `Bean Validation` sobre los DTO) y delega en el Service correspondiente.
- Service: contiene toda la lógica de negocio y las reglas de validación. Es la capa donde se toman decisiones.
- Repository: ejecuta las consultas SQL contra PostgreSQL mediante `JdbcTemplate`.
- DTO (Data Transfer Object): estructuras de transporte de datos entre capas y hacia/desde el frontend. Se dividen en `RequestDTO` (datos de entrada) y `ResponseDTO` (datos de salida).

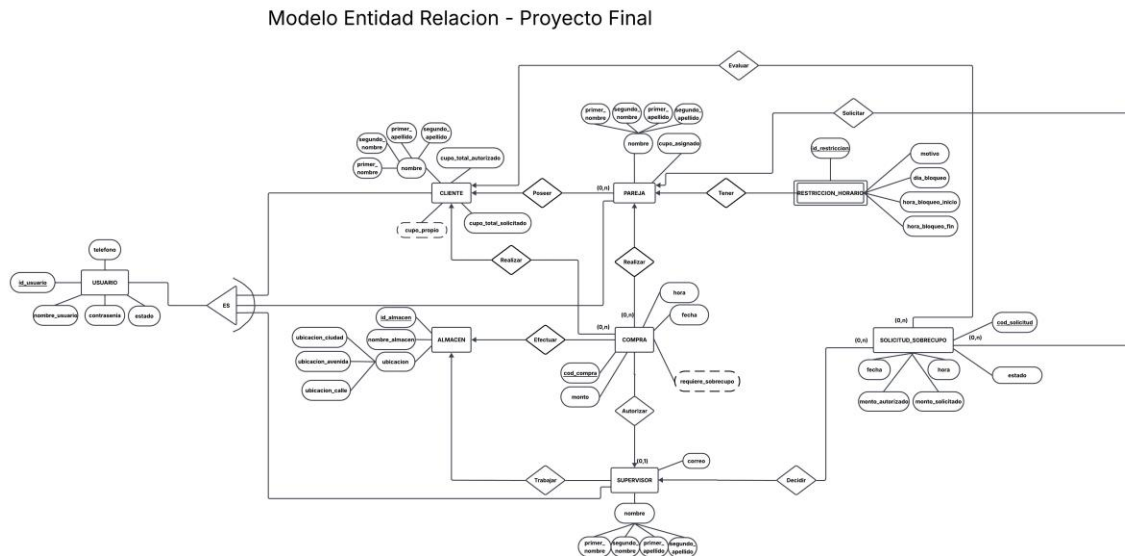
La comunicación entre frontend y backend usa JSON en formato `snake_case` (por ejemplo, `"id_usuario"`, `"primer_nombre"`), configurado mediante la propiedad `"spring.jackson.property-`

naming-strategy=SNAKE_CASE" en el backend, de forma que las clases Java pueden seguir la convención camelCase de Java internamente.

El acceso entre el frontend (puerto 4200) y el backend (puerto 8585) requiere configuración de CORS, definida en la clase "CorsConfig" del backend, que autoriza explícitamente el origen "http://localhost:4200".

3. Modelo Entidad-Relación (MER)

El siguiente diagrama representa las entidades del sistema y sus relaciones.



Las entidades CLIENTE, PAREJA y SUPERVISOR comparten un conjunto de atributos comunes de usuario (identificador, nombre de usuario, contraseña, estado, nombres y apellidos), representados en el diagrama mediante la generalización "ES" hacia una superentidad USUARIO. En la implementación de la base de datos, esta generalización se materializó como tres tablas independientes (CLIENTE, PAREJA, SUPERVISOR), cada una con sus propios atributos particulares, en lugar de una tabla USUARIO física separada.

4. Tecnologías utilizadas

Tecnología	Versión	Uso
Angular	21	Framework principal del frontend (SPA).
TypeScript	incluido con Angular	Lenguaje del frontend.
Node.js	24.14.0	Entorno de ejecución de JavaScript.
Angular CLI	21.2.2	Compilación y ejecución del proyecto Angular.
Spring Boot	4.1.0	Framework principal del backend.
Java JDK	compatible con el proyecto	Lenguaje del backend.
Maven	3.9.16	Gestor de dependencias del backend.
PostgreSQL	18	Motor de base de datos.
pgAdmin 4	—	Administración de la base de datos.
VsCode o IntelliJ	—	Entorno de desarrollo.

El backend NO utiliza un framework ORM (no hay JPA/Hibernate). El acceso a datos se realiza con JdbcTemplate y sentencias SQL explícitas en la capa Repository.

5. Estructura del proyecto

Backend (API)

Organizado por capas, siguiendo el patrón Controller - Service - Repository - DTO - Modelo:

- controlador: clases @RestController (uno por entidad principal: Cliente, Pareja, Supervisor, Almacen, Compra, RestriccionHorario, SolicitudSobrecupo, Login).
- servicio: interfaces de Service.
- servicio/implementacion: implementaciones de los Service, donde vive la lógica de negocio.
- repositorio: clases Repository con las consultas SQL (JdbcTemplate).
- modelo/dto: clases RequestDTO y ResponseDTO.
- modelo: entidades de dominio.
- modelo/excepciones: excepciones custom (RecursoNoEncontradoException, ReglaNegocioException).
- config: configuración (CorsConfig).

- `GlobalExceptionHandler`: manejador centralizado de excepciones, traduce las excepciones de negocio a respuestas HTTP con el código adecuado (404, 400, 500).

Frontend (VIEW)

Organizado bajo el patrón de Diseño Atómico (Atomic Design):

- `components/atoms`: componentes mínimos reutilizables (botón, campo de entrada, badge de estado).
- `components/molecules`: combinaciones simples de átomos (campo de formulario con etiqueta).
- `components/organisms`: bloques funcionales completos (formularios de login/registro, barra lateral, barra de navegación).
- `components/templates`: plantillas de layout compartidas entre páginas (diseno-admin para el Supervisor, diseno-auth para login/registro, diseno-usuario para Cliente/Pareja).
- `pages`: una carpeta por pantalla de la aplicación.
- `services`: un servicio Angular por entidad, encargado exclusivamente de las llamadas HTTP a la API.
- `models/model.ts`: todas las interfaces TypeScript que reflejan los DTO del backend.
- `guards/auth-guard.ts`: protege las rutas que requieren sesión iniciada.

6. Documentación de la API (endpoints)

Todos los endpoints tienen como base "http://localhost:8585/compra". A continuación se listan agrupados por controlador.

LoginController — /api/auth

Método	Ruta	Descripción
POST	/login	Autentica un Cliente, Pareja o Supervisor por documento y contraseña. Devuelve los datos del usuario y su tipo.

SupervisorController — /api/supervisores

Método	Ruta	Descripción
POST	/	Crea un Supervisor.
GET	/ {id}	Obtiene un Supervisor por documento.

GET	/	Lista todos los Supervisores.
GET	/almacen/{idAlmacen}	Lista los Supervisores de un Almacén.
PUT	/id	Actualiza un Supervisor.
DELETE	/id	Elimina un Supervisor.

AlmacenController — /api/almacenes

Método	Ruta	Descripción
POST	/	Crea un Almacén.
GET	/id	Obtiene un Almacén por identificador.
GET	/	Lista todos los Almacenes.

ClienteController — /api/clientes

Método	Ruta	Descripción
POST	/	Crea un Cliente directamente (uso interno del Supervisor/administración).
POST	/registro	Autoregistro público de un Cliente nuevo. Queda en estado "Pendiente", cupo_propio=0, cupo_total_autorizado=0.
GET	/id	Obtiene un Cliente por documento.
GET	/	Lista todos los Clientes.
GET	/pendientes	Lista los Clientes en estado "Pendiente", para la pantalla de Aprobación Cupo Inicial.
PUT	/id	Actualiza los datos de un Cliente.
PUT	/id/aprobar-cupo-inicial	El Supervisor aprueba o edita el cupo inicial de un Cliente Pendiente; pasa a "Activo".
PUT	/id/cupo-propio	Edita puntualmente el cupo propio de un Cliente.
DELETE	/id	Elimina un Cliente.

ParejaController — /api/parejas

Método	Ruta	Descripción
POST	/	Registra una Pareja nueva; transfiere cupo desde el cupo_propio del Cliente titular.
GET	/id	Obtiene una Pareja por documento.

GET	/	Lista todas las Parejas.
GET	/cliente/{idUsuarioCliente}	Lista las Parejas de un Cliente.
PUT	/ {id}	Actualiza los datos de una Pareja.
PUT	/ {id}/inactivar	Cambia el estado de una Pareja a "Inactivo".
PUT	/ {id}/activar	Cambia el estado de una Pareja a "Activo".
DELETE	/ {id}	Elimina una Pareja.

CompraController — /api/compras

Método	Ruta	Descripción
POST	/	Registra una compra (de un Cliente o de una Pareja, arco exclusivo); valida restricciones horarias y cupo disponible.
GET	/ {id}	Obtiene una compra por código.
GET	/	Lista todas las compras.
GET	/pareja/{idUsuarioPareja}	Lista las compras realizadas por una Pareja.
GET	/cliente/{idUsuarioCliente}	Lista las compras de un Cliente, incluyendo las de todas sus Parejas.

RestriccionHorarioController — /api/restricciones-horario

Método	Ruta	Descripción
POST	/	Crea una restricción horaria para una Pareja.
GET	/ {id}	Obtiene una restricción por identificador.
GET	/	Lista todas las restricciones.
GET	/pareja/{idUsuarioPareja}	Lista las restricciones de una Pareja.
PUT	/ {id}	Actualiza una restricción.
DELETE	/ {id}	Elimina una restricción.

SolicitudSobrecupoController — /api/solicitudes-sobrecupo

Método	Ruta	Descripción
POST	/	Una Pareja solicita un sobrecupo. Queda en estado "pendiente_cliente".
GET	/ {id}	Obtiene una solicitud por código.

GET	/	Lista todas las solicitudes.
GET	/cliente/{idUsuarioCliente}	Lista las solicitudes de un Cliente.
GET	/pareja/{idUsuarioPareja}	Lista las solicitudes hechas por una Pareja.
GET	/pendientes-cliente/{idUsuarioCliente}	Lista las solicitudes en "pendiente_cliente" de un Cliente.
GET	/pendientes-supervisor/{idUsuarioSupervisor}	Lista las solicitudes en "pendiente_supervisor" de un Supervisor.
PUT	/ {id} /decision-cliente	El Cliente decide: "Aprobar" (Caso 1), "Escalar" (Caso 2) o "Rechazar" (Caso 3).
PUT	/ {id} /decision-supervisor	El Supervisor aprueba o rechaza una solicitud escalada (Casos 2 y 4).

7. Reglas de negocio implementadas

Estados del Cliente

Estado	Significado
Pendiente	Recién autoregistrado. cupo_propio y cupo_total_autorizado en 0.
Activo	Ya aprobado por el Supervisor. Puede operar normalmente.
Inactivo	Cliente deshabilitado.

Aprobación de Cupo Inicial

Un Cliente nuevo se autoregistra solicitando un "cupo_total_solicitado". Queda en estado "Pendiente" sin cupo operativo. El Supervisor revisa la solicitud desde la pantalla "Aprobación Cupo Inicial" y puede aprobar el monto solicitado tal cual, o editarlo (incluso a \$0, que cumple la función de un rechazo sin necesitar un estado separado). Al aprobar, ese monto final se asigna simultáneamente a cupo_propio y a cupo_total_autorizado del Cliente, y su estado pasa a "Activo". No existe un endpoint de rechazo explícito.

Estados de la Solicitud de Sobrecupo

Estado	Caso de negocio	Descripción
pendiente_cliente	Inicio	Una Pareja solicitó sobrecupo; espera decisión del Cliente.
aprobada_directa	Caso 1	El Cliente aprobó usando su propio cupo_propio.

pendiente_supervisor	Casos 2 y 4	El Cliente escaló la solicitud a un Supervisor (paso intermedio).
aprobada_supervisor	Caso 2	El Supervisor aprobó el incremento de cupo.
rechazada_cliente	Caso 3	El Cliente rechazó la solicitud.
rechazada_supervisor	Caso 4	El Supervisor rechazó la solicitud escalada.

Los cuatro casos de negocio de una solicitud de sobrecupo son:

- Caso 1: la Pareja solicita sobrecupo y el Cliente decide asignarlo desde su cupo propio. No pasa por Supervisor.
- Caso 2: la Pareja solicita sobrecupo, el Cliente decide NO usar su cupo propio, aprueba escalando la solicitud, y el Supervisor la aprueba. El monto se asigna automáticamente a la Pareja y se suma al cupo_total_autorizado del Cliente.
- Caso 3: la Pareja solicita sobrecupo y el Cliente la rechaza directamente.
- Caso 4: igual que el Caso 2, pero el Supervisor rechaza la solicitud escalada.

Arco exclusivo en COMPRA

Una compra siempre pertenece exactamente a un Cliente o a una Pareja, nunca a ambos ni a ninguno. Esta regla está garantizada tanto por un CHECK a nivel de base de datos como por la validación del Service al registrar la compra.

Restricciones horarias

Antes de registrar una compra realizada por una Pareja, el Service valida que no exista una restricción horaria activa para esa fecha y ese rango de horas.

Cálculos derivados (no almacenados, calculados en el Service)

- cupo_gastado y cupo_disponible de una Pareja: se calculan sumando el monto de sus compras y restándolo de cupo_asignado.
- cupo_asignado_parejas de un Cliente: suma de cupo_asignado de todas sus Parejas.
- cupo_total_disponible de un Cliente: $\text{cupo_total_autorizado} - \text{cupo_propio} - \text{cupo_asignado_parejas}$.

8. Instalación y configuración del entorno de desarrollo

Requisitos previos

- PostgreSQL 18 y pgAdmin 4.
- Java JDK (versión compatible con Spring Boot 4.1.0).
- Maven (o el wrapper incluido en el proyecto, API/.mvn).
- Node.js 24.x y npm 11.x.
- Angular CLI 21.x.

Base de datos

Crear una base de datos llamada "Compra" en PostgreSQL y ejecutar el de la base de datos desde el Query Tool de pgAdmin 4. Este script crea las siete tablas del sistema.

Backend

En "API/src/main/resources/application.properties" configurar la URL de conexión, usuario y contraseña de :

spring.datasource.url=jdbc:postgresql://127.0.0.1:5432/Compra

spring.datasource.username=postgres

spring.datasource.password=<contraseña local>

Luego, desde la carpeta "API", ejecutar "mvn spring-boot:run". El servidor queda escuchando en el puerto 8585, con contexto raíz "/compra".

Frontend

Desde la carpeta "VIEW", ejecutar "npm install" (solo la primera vez) y luego "npx ng serve". La aplicación queda disponible en <http://localhost:4200>.